แบบฝึกหัดเตรียมสอบ Final

Parallel Algorithms with CUDA

โจทย์ 4 ส่วน: PRAM | Work-Step | Building Blocks | CUDA Implementation & Fine-tuning

(เฉลยอยู่ท้ายเล่ม — ลองทำเองก่อนเปิดดู)

คำแนะนำ

- ให้ $T^*(n)$ แทนเวลาของ optimal sequential algorithm และ log หมายถึง $\log_2$
- เมื่อโจทย์ให้ออกแบบ WS algorithm ให้เขียน **WS pseudocode** (ใช้ `for all ... in parallel`) พร้อมวิเคราะห์ **Work** $W(n)$ และ **Step** $S(n)$ และบอกว่า work-optimal หรือไม่
- เมื่อโจทย์ให้เสนอ idea fine-tune ให้ตอบเป็น: *ปัญหา* $\rightarrow$ *principle* $\rightarrow$ *วิธีแก้* $\rightarrow$ *วัดผลอย่างไร*
- สัญลักษณ์ **[Quiz]** หมายถึงโจทย์ที่ดัดแปลงจากการบ้าน/ควิซในโฟลเดอร์ `homework`

Contents

1 ส่วน A: PRAM Model และ Optimal Speedup **2**

2 ส่วน B: Work-Step Model **4**

3 ส่วน C: Basic Building Blocks และการ Decompose ปัญหา **9**

4 ส่วน D: CUDA Implementation และ Fine-tuning **15**

5 เฉลย **23**

1 ส่วน A: PRAM Model และ Optimal Speedup

ข้อ A1 นิยามพื้นฐาน

(a) จงนิยาม speedup, cost และ efficiency ของ PRAM algorithm และบอกเงื่อนไขของ *optimal speedup*

(b) จงอธิบายว่าทำไม speedup ของ deterministic parallel algorithm บน p processors จึงไม่สามารถเกิน $O(p)$ ได้

พื้นที่ทำ

ข้อ A2 [Quiz] คูณจำนวนเต็ม n จำนวน

กำหนดจำนวนเต็ม n จำนวน ต้องการหาผลคูณของทุกจำนวน (สมมุติว่าการคูณหนึ่งครั้งใช้เวลา $O(1)$)

(a) จงอธิบาย optimal sequential algorithm พร้อมระบุ running time และให้เหตุผลว่าทำไมจึง optimal

(b) จงอธิบาย optimal parallel algorithm บน PRAM ระบุ running time, จำนวน processors และ PRAM model ที่ต้องใช้ พร้อมให้เหตุผลว่าทำไมจึง optimal

พื้นที่ทำ

ข้อ A3 คำนวณ speedup เป็นตัวเลข

ต้องการหาผลรวมของ $n = 1024$ จำนวนบน EREW PRAM โดยให้แต่ละ processor รวม block ของตัวเองแบบ sequential แล้วรวมผลด้วย binary tree (นับ 1 การบวก $=$ 1 หน่วยเวลา; sequential ใช้ $n - 1$ การบวก) จงคำนวณเวลา T_p, speedup และ efficiency เมื่อ

(a) $p = 32$ (b) $p = 128$ (c) $p = 1024$

แล้วสรุปว่าการเพิ่ม processor ส่งผลต่อ efficiency อย่างไร และสอดคล้องกับเงื่อนไข $p = O(n/\log n)$ อย่างไร

พื้นที่ทำ

ข้อ A4 ช่วงของ p ที่ให้ optimal speedup

ปัญหาหนึ่งมี $T^*(n) = \Theta(n \log n)$ และมี PRAM algorithm ที่ใช้เวลา $T_p(n) = O\left(\frac{n \log n}{p} + \log^2 n\right)$

(a) จงหา cost ของ algorithm

(b) p ต้องมีค่าอย่างมากเท่าใด (ในรูป asymptotic) จึงจะได้ optimal speedup

(c) ถ้าใช้ $p = n$ จะได้ speedup เท่าใด เป็น optimal speedup หรือไม่

พื้นที่ทำ

ข้อ A5 พลังของ Concurrent Write

(a) จงอธิบาย algorithm หา maximum ของ n จำนวนใน $O(1)$ step บน Common CRCW PRAM ระบุจำนวน processors และ work

(b) ทำไมการเขียนพร้อมกันใน algorithm ข้อ (a) จึงถูกต้องตามกติกาของ *Common* CRCW

(c) algorithm ข้อ (a) work-optimal หรือไม่ และบน EREW PRAM จะหา maximum ได้เร็วที่สุดกี่ step เพราะอะไร

พื้นที่ทำ

2 ส่วน B: Work-Step Model

ข้อ B1 [Quiz] ลักษณะของ WS algorithm ที่ดี

(a) ลักษณะสำคัญของ parallel solution ที่ดีบน WS model มีอะไรบ้าง จงอธิบายว่าทำไมจึงสำคัญ

(b) ระหว่าง WS algorithm สองตัว: ตัวแรกมี work $O(T^*(n))$ และ step $O(\sqrt{n})$; ตัวที่สองมี work $O(T^*(n)\log n)$ และ step $O(\log n)$ ตัวไหนดีกว่า จงให้เหตุผล

(c) ระหว่างตัวที่มี work $O(T^*(n)\log n)$, step $\log n$ กับตัวที่มี work $O(T^*(n))$, step $2\log n$ ตัวไหนดีกว่า

(d) เป็นไปได้หรือไม่ที่จะออกแบบ WS algorithm ที่มี work $o(T^*(n))$ จงให้เหตุผล

พื้นที่ทำ

ข้อ B2 [Quiz] หา Maximum ด้วย WS algorithm

(a) ถ้า algorithm ที่ออกแบบเป็น optimal แล้ว work และ step complexity ควรเป็นเท่าใด

(b) จงเขียน WS pseudocode ของ optimal WS algorithm (สมมุติ $n = 2^k$)

(c) ใช้ Brent's theorem หาเวลาเมื่อรันบน p processors และช่วงของ p ที่ได้ optimal speedup

พื้นที่ทำ

ข้อ B3 วิเคราะห์ WS pseudocode ที่กำหนดให้

Algorithm 1 Algorithm X

```
for d = 1 to log n do
    for all 0 ≤ k < n in parallel do
        if k ≥ 2^(d−1) then
            x[k] ← x[k − 2^(d−1)] + x[k]
        end if
    end for
end for
```

(a) Algorithm X คำนวณอะไร

(b) จงหา $W(n)$ (นับจำนวนการบวกให้ละเอียด) และ $S(n)$

(c) Algorithm X work-optimal หรือไม่ ถ้ารันบน p processors จะได้ optimal speedup ได้หรือไม่

(d) ถ้า implement ใน CUDA แบบ in-place ตรง ๆ จะเกิดปัญหาอะไร

พื้นที่ทำ

ข้อ B4 ออกแบบ WS algorithm: Inner product

กำหนดเวกเตอร์ x, y ยาว n จงออกแบบ WS algorithm คำนวณ $x \cdot y = \sum_i x_i y_i$ เขียน WS pseudocode, วิเคราะห์ $W(n)$, $S(n)$ และบอก PRAM model ที่ต้องใช้

พื้นที่ทำ

ข้อ B5 Algorithm cascading สำหรับ Sum

(a) จงเขียน WS pseudocode ของ sum ที่แบ่ง input เป็น block ขนาด $\log n$ ทำ sequential ในแต่ละ block ก่อน แล้วจึงใช้ balanced tree

(b) วิเคราะห์ $W(n)$ และ $S(n)$ แล้วเปรียบเทียบกับ tree reduce ธรรมดา

(c) แนวคิดนี้สัมพันธ์กับเทคนิคใดใน CUDA

พื้นที่ทำ

ข้อ B6 Pointer jumping: List ranking

กำหนด linked list n node เก็บใน array `next[]` (node สุดท้ายชี้ตัวเอง) ต้องการหา `rank[i]` = ระยะทางจาก node i ถึงปลาย list

Algorithm 2 List ranking

1: **for all** $0 \le i < n$ **in parallel do**
2: $\quad rank[i] \leftarrow$ (**if** $next[i] = i$ **then** 0 **else** 1)
3: **end for**
4: **for** $t = 1$ **to** $\lceil \log n \rceil$ **do**
5: $\quad$ **for all** $0 \le i < n$ **in parallel do**
6: $\quad\quad rank[i] \leftarrow rank[i] + rank[next[i]]; \quad next[i] \leftarrow next[next[i]]$
7: $\quad$ **end for**
8: **end for**

(a) จงหา $W(n)$ และ $S(n)$ และบอกว่า work-optimal หรือไม่ ($T^*(n) = \Theta(n)$)

(b) ต้องใช้ PRAM model อะไร (EREW หรือ CREW) เพราะอะไร

(c) บน p processors ได้ speedup อย่างมากเท่าใด

พื้นที่ทำ

ข้อ B7 Recurrence ของ Divide-and-conquer

Parallel merge sort มี $W(n) = 2W(n/2) + O(n)$ และ $S(n) = S(n/2) + O(\log n)$ (merge ใช้ $O(\log n)$ step)

(a) จงแก้ recurrence หา $W(n)$ และ $S(n)$

(b) work-optimal หรือไม่ (comparison sort มี $T^*(n) = \Theta(n \log n)$)

(c) จำนวน processors สูงสุดที่ยังได้ optimal speedup คือเท่าใด

พื้นที่ทำ

ข้อ B8 Matrix–vector multiplication

จงออกแบบ WS algorithm คำนวณ $y = Ax$ เมื่อ A เป็น matrix ขนาด $n \times n$ เขียน WS pseudocode และวิเคราะห์ $W(n)$, $S(n)$ พร้อมบอกว่า work-optimal หรือไม่

พื้นที่ทำ

3 ส่วน C: Basic Building Blocks และการ Decompose ปัญหา

ข้อ C1 Trace การทำ Scan

กำหนด $A = \langle 2, 5, 1, 3, 4, 0, 6, 2 \rangle$ และ operator $+$

(a) จงหา inclusive scan และ exclusive scan

(b) จงแสดงค่าของ array หลังแต่ละ step ของ **Hillis–Steele** scan

(c) จงแสดงค่าของ array หลังแต่ละ step ของ **Blelloch** scan (up-sweep และ down-sweep)

(d) นับจำนวนการบวกของ (b), (c) เทียบกับ sequential scan

พื้นที่ทำ

ข้อ C2 Stream compaction

ต้องการลบค่าติดลบออกจาก $A = \langle 3, -1, 4, -1, -5, 9, 2, -6 \rangle$ โดยรักษาลำดับเดิม จง decompose เป็น building blocks แสดงค่ากลางทุกขั้น และวิเคราะห์ W, S

พื้นที่ทำ

ข้อ C3 ตรวจวงเล็บสมดุล

กำหนดสตริงวงเล็บความยาว n จงออกแบบ parallel algorithm ตรวจว่าวงเล็บสมดุล (balanced) หรือไม่ โดยใช้ building blocks แล้วทดสอบกับ `(()(()))` และ `())(()`

พื้นที่ทำ

ข้อ C4 Linear recurrence ด้วย Scan

ต้องการคำนวณ $x_i = a_i x_{i-1} + b_i$ สำหรับ $i = 1, \ldots, n$ โดยกำหนด x_0

(a) จงนิยาม operator $\otimes$ บนคู่ (a, b) ที่ทำให้ใช้ scan แก้ปัญหานี้ได้ และหา identity

(b) พิสูจน์ว่า $\otimes$ associative; $\otimes$ commutative หรือไม่

(c) ทดสอบกับ $x_0 = 1$, $(a_1, b_1) = (2, 1)$, $(a_2, b_2) = (3, 0)$, $(a_3, b_3) = (1, 4)$

พื้นที่ทำ

ข้อ C5 Median ของคะแนน

นักศึกษา n คนมีคะแนนเป็นจำนวนเต็ม 0–100 จงหา median ด้วย building blocks (ไม่ต้อง sort ทั้ง array) พร้อมวิเคราะห์ work และ step

พื้นที่ทำ

ข้อ C6 Split และ Radix sort

กำหนด $A = \langle 5, 2, 7, 4, 1, 6 \rangle$ (3 bits)

(a) ทำ split ตาม bit 0 (LSB) โดยแสดง flag F, `Down = enumerate(not F)`, `P = enumerate(F)` และ index ปลายทาง

(b) ทำ radix sort ให้ครบทุก bit แสดงผลแต่ละรอบ

(c) ทำไม split ต้องเป็น *stable*

พื้นที่ทำ

ข้อ C7 Line of sight

ผู้สังเกตอยู่ที่ตำแหน่ง 0 ที่ระดับความสูง h_0 จุดที่ i อยู่ห่าง i หน่วย สูง h_i จุด i จะ *มองเห็นได้* ถ้ามุมเงย θ_i มากกว่ามุมเงยของทุกจุด $j < i$

(a) จง decompose เป็น building blocks พร้อมวิเคราะห์ W, S

(b) ทดสอบกับ $h = \langle 10, 12, 11, 15, 14, 22, 26 \rangle$ (ใช้ $\tan\theta_i = (h_i - h_0)/i$)

พื้นที่ทำ

ข้อ C8 Run-length encoding

จง decompose ปัญหา run-length encoding เป็น building blocks แล้วทดสอบกับ $A = \langle a, a, b, b, b, c, a, a \rangle$ (คาดหวังผล: ค่า $\langle a, b, c, a \rangle$, ความยาว $\langle 2, 3, 1, 2 \rangle$)

พื้นที่ทำ

ข้อ C9 Polynomial evaluation

จงคำนวณ $p(x) = \sum_{i=0}^{n-1} a_i x^i$ แบบขนานด้วย building blocks แล้วทดสอบกับ $a = \langle 3, 0, 1, 2 \rangle$, $x = 2$

พื้นที่ทำ

ข้อ C10 Mean และ Variance ใน Reduce เดียว

จงออกแบบ operator สำหรับ reduce ครั้งเดียวที่ให้ทั้ง mean และ variance ของ n จำนวน พิสูจน์ว่าเป็น associative และบอกข้อควรระวังเชิงตัวเลข

พื้นที่ทำ

4 ส่วน D: CUDA Implementation และ Fine-tuning

ข้อ D1 [Practice 4.1] Reduce ด้วย global memory อย่างเดียว

(a) จงเขียน CUDA kernel และ host loop สำหรับ parallel sum โดยใช้ global memory อย่างเดียว (รองรับ n ที่ไม่ใช่กำลังของ 2)

(b) ถ้า $n = 2^{20}$ ต้อง launch kernel กี่ครั้ง ทำไมจึงต้อง launch หลายครั้งแทนการใช้ `__syncthreads()`

(c) ประมาณปริมาณ global memory traffic ทั้งหมด (bytes) เทียบกับ version ที่ใช้ shared memory

พื้นที่ทำ

ข้อ D2 [Practice 4.2] วิเคราะห์ shared-memory reduce

```
__global__ void reduce0(const float *in, float *out) {
    extern __shared__ float sdata[];
    unsigned tid = threadIdx.x;
    unsigned i = blockIdx.x * blockDim.x + tid;
    sdata[tid] = in[i];
    __syncthreads();
    for (unsigned s = 1; s < blockDim.x; s *= 2) {
        if (tid % (2 * s) == 0) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }
    if (tid == 0) out[blockIdx.x] = sdata[0];
}
```

(a) ที่ `blockDim.x = 256` ใน iteration แรก ($s = 1$) มี thread ทำงานกี่ตัว และกี่ warp ต้อง execute

(b) ระบุปัญหาด้าน performance ของ kernel นี้อย่างน้อย 3 ข้อ

(c) จงเขียน kernel ที่ปรับปรุงแล้ว และอธิบายว่าแต่ละการเปลี่ยนแปลงอิง principle อะไร

พื้นที่ทำ

ข้อ D3 Effective bandwidth

GPU มี peak memory bandwidth 900 GB/s

(a) Reduce 2^{24} floats ใช้เวลา 0.5 ms ได้ effective bandwidth เท่าใด คิดเป็นกี่ % ของ peak

(b) หลัง optimize เหลือ 0.085 ms ได้กี่ GB/s และกี่ %

(c) Scan 2^{24} floats (อ่านครั้งเดียว เขียนครั้งเดียว) ใช้เวลา 0.4 ms ได้กี่ % ของ peak

(d) ทำไมเราจึงวัด reduce/scan ด้วย GB/s แทน GFLOP/s (ใช้แนวคิด arithmetic intensity)

พื้นที่ทำ

ข้อ D4 [Practice 5.1] หา bug ใน Hillis–Steele kernel

```
__global__ void scanBug(float *x, int n) {   // n == blockDim.x, single block
    extern __shared__ float t[];
    int tid = threadIdx.x;
    t[tid] = x[tid];
    __syncthreads();
    for (int off = 1; off < n; off *= 2) {
        if (tid >= off) t[tid] += t[tid - off];
        __syncthreads();
    }
    x[tid] = t[tid];
}
```

จงอธิบายว่า bug คืออะไร เกิดเมื่อไร และแก้ไขได้อย่างน้อย 2 วิธี

พื้นที่ทำ

ข้อ D5 [Practice 5.2] Scan ขนาดใหญ่หลาย block

ต้องการ exclusive scan $n = 2^{22}$ elements โดย Blelloch kernel หนึ่ง block scan ได้ 1024 elements

(a) จงอธิบายขั้นตอน hierarchical scan และนับจำนวน kernel launch ทั้งหมด

(b) ประมาณ global memory traffic (เป็นจำนวนเท่าของ n) และเสนอแนวทางลด traffic

พื้นที่ทำ

ข้อ D6 [Practice 7.2] Privatized histogram

(a) จงเขียน CUDA kernel histogram ของ ASCII 128 bins ที่ใช้ private histogram ใน shared memory ต่อ block

(b) ถ้า input มี 10^8 ตัวอักษร ใช้ 120 blocks จงเปรียบเทียบจำนวน global atomic ของ naive กับ privatized

(c) ถ้าข้อมูลเป็นข้อความภาษาอังกฤษ (ตัวอักษรบางตัวพบบ่อยมาก) ทำไม naive version จึงช้าเป็นพิเศษ และเสนอ fine-tune เพิ่มเติม

พื้นที่ทำ

ข้อ D7 [Practice 7.1] Histogram 4096 bins แบบ saturate ที่ 127

input เป็นจำนวนเต็มในช่วง $[0, 4096)$ ยาวได้ถึง 2^{32} ตัว counter เป็น unsigned 32-bit และต้อง saturate ที่ 127 (ห้ามวนกลับเป็น 0)

(a) private histogram ต่อ block ใช้ shared memory เท่าใด

(b) ถ้า SM มี shared memory 64 KB และ 2048 threads สูงสุด ใช้ block 256 threads ได้ occupancy เท่าใด ควรปรับอย่างไร

(c) การ "histogram ปกติแล้วค่อย clamp ตอนท้าย" มีกับดักอะไรกับ input ขนาดนี้ จงออกแบบวิธีที่ถูกต้อง

พื้นที่ทำ

ข้อ D8 คำนวณ Occupancy

SM หนึ่งตัวรองรับ: threads สูงสุด 2048, blocks สูงสุด 32, registers 65,536, shared memory 64 KB จงหาจำนวน block ต่อ SM, ข้อจำกัดที่เป็นคอขวด และ occupancy ของแต่ละกรณี

กรณี	threads/block	registers/thread	shared/block
(a)	256	32	8 KB
(b)	256	64	8 KB
(c)	256	32	24 KB
(d)	32	16	0

พื้นที่ทำ

ข้อ D9 Memory coalescing

Warp หนึ่ง (32 threads) อ่าน `float` จาก global memory (array align 128 B, transaction ละ 32 B sector) thread i ($0 \le i < 32$) อ่านตำแหน่งต่อไปนี้ จงหาจำนวน sector ที่ต้องโหลดและ % efficiency

(a) $A[i]$ (b) $A[2i]$ (c) $A[32i]$ (d) $A[i+1]$

(e) ถ้าเก็บข้อมูลเป็น array of structs `struct {float x,y,z;}` และ kernel อ่านเฉพาะ `.x` จะได้ efficiency เท่าใด เสนอวิธีแก้

พื้นที่ทำ

ข้อ D10 Shared memory bank conflicts

Shared memory มี 32 banks กว้าง 4 bytes; thread t ใน warp เข้าถึง `float sh[]` ที่ index ต่อไปนี้ จงบอกว่าเกิด bank conflict กี่ทาง (n-way)

(a) $sh[t]$ (b) $sh[2t]$ (c) $sh[32t]$ (d) $sh[33t]$

และอธิบายว่าเหตุใด Blelloch scan ใน shared memory จึงเกิด bank conflict และแก้อย่างไร

พื้นที่ทำ

ข้อ D11 ออกแบบ End-to-end: กรองค่าที่สูงกว่าค่าเฉลี่ย

กำหนดค่าจาก sensor n ค่า (float) ต้องการ output เฉพาะค่าที่มากกว่าค่าเฉลี่ย โดยรักษาลำดับเดิม

(a) จง decompose เป็น building blocks และวิเคราะห์ W, S (high-level)

(b) จงออกแบบ CUDA kernels (ระบุว่ามีกี่ kernel แต่ละตัวทำอะไร)

(c) เสนอ idea fine-tune อย่างน้อย 3 ข้อ โดยอิงหลักการที่วัดได้ (throughput / bandwidth)

พื้นที่ทำ

5 เฉลย

ส่วน A: PRAM

เฉลยข้อ A1

(a) ให้ $T_p(n)$ คือเวลาบน p processors: speedup $S_p = T^*(n)/T_p$; cost $C_p = pT_p$; efficiency $E_p = S_p/p = T^*(n)/C_p$. **Optimal speedup** คือ $S_p = \Theta(p)$ ซึ่งสมมูลกับ cost-optimal $C_p = \Theta(T^*(n))$ หรือ $E_p = \Theta(1)$

(b) ถ้า $S_p = \omega(p)$ แล้ว processor เดียวสามารถ simulate p processors ทีละขั้น ใช้เวลา $p \cdot T_p = p \cdot T^*(n)/S_p = o(T^*(n))$ จึงได้ sequential algorithm ที่เร็วกว่า optimal — ขัดแย้งกับนิยามของ $T^*(n)$

เฉลยข้อ A2

(a) **Sequential**: วน loop คูณสะสม `r = 1; for i: r = r * a[i]` ใช้ $n-1$ การคูณ $\Rightarrow T^*(n) = \Theta(n)$, ใช้ memory $O(1)$ เพิ่มเติม. Optimal เพราะคำตอบขึ้นกับทุก input ต้องอ่าน input ทั้ง n ตัวอย่างน้อย $\Rightarrow$ lower bound $\Omega(n)$

(b) **Parallel (EREW PRAM)** ใช้ $p = n/\log n$ processors:

1. processor i คูณ block ของตัวเอง ($\log n$ ตัว) แบบ sequential: $O(\log n)$
2. คูณผล p ค่าด้วย balanced binary tree: $O(\log p) = O(\log n)$

$T_p = O(\log n)$, cost $= pT_p = (n/\log n) \cdot O(\log n) = O(n) = O(T^*(n)) \Rightarrow$ cost-optimal speedup $= \Theta(n/\log n) = \Theta(p)$. ไม่มีการอ่าน/เขียนตำแหน่งเดียวกันพร้อมกัน จึงใช้ EREW ได้. นอกจากนี้ step $O(\log n)$ ก็เป็น minimum แล้ว เพราะการคูณเป็น binary operation (fan-in 2) ต้องใช้อย่างน้อย $\lceil \log n \rceil$ step จึงจะรวม input n ตัวเข้าสู่ output เดียว

เฉลยข้อ A3

$T_p = (n/p - 1) + \log p$; sequential $= 1023$

p	block size	local	tree	T_p	speedup / efficiency
32	32	31	5	36	$1023/36 \approx 28.4$, $E \approx 0.89$
128	8	7	7	14	$1023/14 \approx 73.1$, $E \approx 0.57$
1024	1	0	10	10	102.3, $E \approx 0.10$

เมื่อเพิ่ม p เกินกว่า $n/\log n = 1024/10 \approx 102$ ส่วน $\log p$ (tree) เริ่มครอบงำ เวลาลดลงไม่มากแต่ processor เพิ่มมาก efficiency จึงตก — ตรงกับเงื่อนไข optimal speedup $p = O(n/\log n)$

เฉลยข้อ A4

(a) $C_p = pT_p = O(n \log n + p \log^2 n)$
(b) ต้องการ $p \log^2 n = O(n \log n) \Rightarrow p = O(n/\log n)$
(c) $p = n$: $T_p = O(\log n + \log^2 n) = O(\log^2 n)$, speedup $= \Theta(n \log n / \log^2 n) = \Theta(n/\log n)$ ซึ่ง $= o(p) \Rightarrow$ **ไม่** optimal (cost $= n \log^2 n$)

เฉลยข้อ A5

(a) ใช้ n^2 processors: (1) `forall` i: $M[i] \leftarrow 1$; (2) `forall` (i, j): ถ้า $A[i] < A[j]$ ให้ $M[i] \leftarrow 0$; (3) `forall` i: ถ้า $M[i] = 1$ ให้ตอบ $A[i]$ — ใช้ $O(1)$ step, work $O(n^2)$
(b) processor ทุกตัวที่เขียน $M[i]$ พร้อมกันเขียนค่า *เดียวกัน* คือ 0 จึงถูกกติกา Common CRCW (ขั้น (3) ถ้ามี maximum ซ้ำหลายตัว ทุกตัวเขียนค่าเดียวกันลง output เช่นกัน)
(c) ไม่ work-optimal (n^2 vs $T^*(n) = n$). บน EREW (และ CREW) ต้องใช้ $\Omega(\log n)$ step เพราะ operator binary มี fan-in 2: หลัง t step แต่ละค่าขึ้นกับ input อย่างมาก 2^t ตัว ต้องการ $2^t \geq n$. (บน CRCW สามารถ cascade ได้ $O(\log \log n)$ step แบบ work-optimal)

ส่วน B: Work-Step

เฉลยข้อ B1

(a) (1) **Work-optimal** $W = \Theta(T^*(n))$ — จาก Brent $T_p = O(W/p + S)$ ถ้า $W = \omega(T^*(n))$ speedup $\leq pT^*(n)/W \to 0$ เมื่อ n โต ในขณะที่ p ในโลกจริงจำกัด; (2) **Step น้อย** (polylog) — ให้ parallelism W/S สูง ได้ optimal speedup ถึง $p = O(T^*(n)/S)$; (3) ใช้ memory model ที่อ่อน (EREW) — implement ได้จริงบน hardware; (4) เขียนเป็น pseudocode ที่ชัดเจน map ลง hardware ได้

(b) **ตัวแรกดีกว่า** เพราะ work-optimal: $T_p = O(T^*(n)/p + \sqrt{n})$ ได้ optimal speedup เมื่อ $p = O(T^*(n)/\sqrt{n})$ ส่วนตัวที่สอง $T_p = O(T^*(n) \log n/p + \log n)$ speedup ไม่เกิน $p/\log n$ ไม่มีทาง optimal และยิ่ง n ใหญ่ยิ่งแย่ (step ที่ต่ำกว่าช่วยเฉพาะเมื่อ p มหาศาลเกิน $T^*(n)/\sqrt{n}$)

(c) **ตัวที่สองดีกว่า**: work-optimal และ step $2 \log n = O(\log n)$ เท่ากับตัวแรกในเชิง asymptotic

(d) **ไม่ได้**: ถ้า $W = o(T^*(n))$ เราสามารถรัน WS algorithm บน processor เดียวในเวลา $O(W)$ (Brent, $p = 1$: $T_1 \leq W + S \leq 2W$) ได้ sequential algorithm ที่เร็วกว่า $T^*(n)$ ซึ่งขัดแย้งว่า $T^*(n)$ optimal

เฉลยข้อ B2

(a) $W(n) = \Theta(n)$, $S(n) = \Theta(\log n)$ (บน EREW/CREW)

Algorithm 3 WS-Max

for all $1 \le i \le n$ **in parallel do**
    $B[i] \leftarrow A[i]$
end for
for $h = 1$ **to** $\log n$ **do**
    for all $1 \le i \le n/2^h$ **in parallel do**
        $B[i] \leftarrow \max(B[2i-1], B[2i])$
    end for
end for
return $B[1]$

$W = n + n/2 + n/4 + \cdots = O(n)$, $S = \log n + O(1)$
(c) Brent: $T_p \le n/p + \log n + O(1) \Rightarrow$ optimal speedup เมื่อ $p = O(n/\log n)$

เฉลยข้อ B3

(a) Hillis–Steele inclusive prefix sum
(b) round d บวก $n - 2^{d-1}$ ครั้ง: $W = \sum_{d=1}^{\log n}(n - 2^{d-1}) = n\log n - (n-1) = \Theta(n \log n)$, $S = \log n$
(c) ไม่ work-optimal ($T^*(n) = n$). $T_p = O(n \log n/p + \log n)$ ให้ speedup $\le p/\log n$ เสมอ $\Rightarrow$ ไม่มี p ใดที่ได้ optimal speedup (เมื่อ n โต)
(d) Race condition: semantics ของ forall ต้องอ่านค่าเก่าทุกตัวก่อนเขียน แต่ thread หนึ่งอาจอ่าน $x[k - 2^{d-1}]$ ที่ถูก thread อื่นอัปเดตไปแล้วใน round เดียวกัน (ดูข้อ D4) — ต้องใช้ double buffer

เฉลยข้อ B4

Algorithm 4 Inner product

for all $1 \le i \le n$ **in parallel do**
    $z[i] \leftarrow x[i] \cdot y[i]$ ▷ map
end for
for $h = 1$ **to** $\log n$ **do** ▷ reduce
    for all $1 \le i \le n/2^h$ **in parallel do**
        $z[i] \leftarrow z[2i-1] + z[2i]$
    end for
end for
return $z[1]$

$W = n + O(n) = O(n) = \Theta(T^*(n))$ (work-optimal), $S = 1 + \log n = O(\log n)$, EREW (แต่ละตำแหน่งถูกอ่าน/เขียนโดย operation เดียวต่อ step)

เฉลยข้อ B5

(a)

Algorithm 5 Cascaded sum

```
1: b ← log n; m ← n/b
2: for all 1 ≤ j ≤ m in parallel do
3:     B[j] ← 0
4:     for k = (j − 1)b + 1 to jb do
5:         B[j] ← B[j] + A[k]                          ▷ sequential
6:     end for
7: end for
8: for h = 1 to log m do
9:     for all 1 ≤ i ≤ m/2^h in parallel do
10:        B[i] ← B[2i − 1] + B[2i]
11:    end for
12: end for
13: return B[1]
```

(b) ช่วงแรก: $W = O(n)$, $S = O(\log n)$; ช่วง tree บน $m = n/\log n$ ค่า: $W = O(n/\log n)$, $S = O(\log n)$ รวม $W = O(n)$, $S = O(\log n)$ — asymptotic เท่า tree ธรรมดา แต่ จำนวน *operation ที่ทำพร้อมกันต่อ step* มีเพียง $n/\log n$ จึงต้องการเพียง $p = n/\log n$ processors ก็ได้ $T_p = O(\log n)$ (tree ธรรมดาถ้าใช้ $p = n$ จะมี cost $n \log n$)
(c) **Thread coarsening** / grid-stride loop: แต่ละ thread รวมหลาย element แบบ sequential ใน register ก่อน แล้วค่อย reduce แบบ tree ใน shared memory/warp shuffle

เฉลยข้อ B6

(a) $S = O(\log n)$ (initial 1 step + $\lceil \log n \rceil$ rounds), $W = n + n\lceil \log n \rceil = \Theta(n \log n) \Rightarrow$ **ไม่** work-optimal
(b) ต้องใช้ **CREW**: หลัง node ใกล้ปลายชี้ไปที่ node สุดท้ายแล้ว node หลายตัวจะอ่าน $rank[next[i]]$ และ $next[next[i]]$ ของ node เดียวกันพร้อมกัน (concurrent read) แต่การเขียนแต่ละตัวเขียนเฉพาะ ตำแหน่ง i ของตัวเอง (exclusive write)
(c) $T_p = O(n \log n/p + \log n) \Rightarrow$ speedup $= O(p/\log n)$ ไม่ optimal

เฉลยข้อ B7

(a) $W(n) = 2W(n/2) + O(n) = O(n \log n)$ (ทุกชั้นของ recursion tree มี work $O(n)$, มี $\log n$ ชั้น); $S(n) = S(n/2) + O(\log n) = O(\log n + \log(n/2) + \cdots) = O(\log^2 n)$
(b) work-optimal เพราะ $W = \Theta(n \log n) = \Theta(T^*(n))$
(c) $p = O(T^*(n)/S) = O(n \log n/\log^2 n) = O(n/\log n)$

เฉลยข้อ B8

Algorithm 6 Matrix–vector

1: **for all** $1 \le i, j \le n$ **in parallel do**
2: $\quad P[i][j] \leftarrow A[i][j] \cdot x[j]$
3: **end for**
4: **for all** $1 \le i \le n$ **in parallel do**
5: $\quad y[i] \leftarrow \text{REDUCE}(P[i][1..n], +)$
6: **end for**

$W = n^2 + n \cdot O(n) = O(n^2) = \Theta(T^*(n))$ (ต้องอ่าน A ทั้ง n^2 ตัว) $\Rightarrow$ work-optimal; $S = 1 + O(\log n) = O(\log n)$ (reduce ทุกแถวทำพร้อมกัน step เป็น max ไม่ใช่ผลรวม). หมายเหตุ: $x[j]$ ถูกอ่านโดย n operation พร้อมกัน จึงเป็น CREW (ถ้าต้องการ EREW ให้ broadcast x ก่อนด้วย $O(\log n)$ step, $O(n^2)$ work)

ส่วน C: Building Blocks

เฉลยข้อ C1

(a) inclusive $= \langle 2, 7, 8, 11, 15, 15, 21, 23 \rangle$; exclusive $= \langle 0, 2, 7, 8, 11, 15, 15, 21 \rangle$
(b) Hillis–Steele:

เริ่มต้น	$2, 5, 1, 3, 4, 0, 6, 2$
offset 1	$2, 7, 6, 4, 7, 4, 6, 8$
offset 2	$2, 7, 8, 11, 13, 8, 13, 12$
offset 4	$2, 7, 8, 11, 15, 15, 21, 23$

(c) Blelloch:

เริ่มต้น	$2, 5, 1, 3, 4, 0, 6, 2$
up $d = 0$	$2, 7, 1, 4, 4, 4, 6, 8$
up $d = 1$	$2, 7, 1, 11, 4, 4, 6, 12$
up $d = 2$	$2, 7, 1, 11, 4, 4, 6, 23$
set $x[7] = 0$	$2, 7, 1, 11, 4, 4, 6, 0$
down $d = 2$	$2, 7, 1, 0, 4, 4, 6, 11$
down $d = 1$	$2, 0, 1, 7, 4, 11, 6, 15$
down $d = 0$	$0, 2, 7, 8, 11, 15, 15, 21$

(d) Sequential: $n - 1 = 7$; Hillis–Steele: $7 + 6 + 4 = 17$ $(= n \log n - (n - 1))$; Blelloch: up $4 + 2 + 1 = 7$, down $1 + 2 + 4 = 7$ รวม 14 $(= 2(n - 1))$ — Blelloch work-efficient, Hillis–Steele ไม่ใช่ (โตเป็น $n \log n$)

เฉลยข้อ C2

1. **Map** flag $F_i = [A_i \ge 0]$: $\langle 1, 0, 1, 0, 0, 1, 1, 0 \rangle$
2. **Exclusive scan**$(+)$ ของ F: $X = \langle 0, 1, 1, 2, 2, 2, 3, 4 \rangle$, total $= 4$

3. **Scatter**: ถ้า $F_i = 1$ ให้ $R[X_i] \leftarrow A_i \Rightarrow R = \langle 3, 4, 9, 2 \rangle$

$W = O(n)$, $S = O(\log n)$ (จาก scan), work-optimal

เฉลยข้อ C3

Map: $($ $\to +1$, $)$ $\to -1 \to$ inclusive scan$(+)$ ได้ "ความลึก" $D \to$ balanced iff **reduce(min)** ของ $D \geq 0$ **และ** $D_{n-1} = 0$. $W = O(n)$, $S = O(\log n)$

- `(()(()))`: $+1, +1, -1, +1, +1, -1, -1, -1 \to D = 1, 2, 1, 2, 3, 2, 1, 0$; min $= 0$, ตัวสุดท้าย $= 0 \Rightarrow$ **balanced**
- `())(()`: $+1, -1, -1, +1, +1, -1 \to D = 1, 0, -1, 0, 1, 0$; min $= -1 < 0 \Rightarrow$ **ไม่ balanced** (แม้ผลรวมจะเป็น 0)

เฉลยข้อ C4

(a) มองแต่ละขั้นเป็น function $f_i(x) = a_i x + b_i$ ใช้ f_1 ก่อนแล้วตามด้วย f_2: $f_2(f_1(x)) = a_2(a_1 x + b_1) + b_2 = (a_1 a_2)x + (a_2 b_1 + b_2)$ จึงนิยาม

$$(a_1, b_1) \otimes (a_2, b_2) = (a_1 a_2,\ a_2 b_1 + b_2), \qquad \text{identity } I = (1, 0)$$

ทำ inclusive scan ได้ (A_i, B_i) แล้ว $x_i = A_i x_0 + B_i$ (map). $W = O(n)$, $S = O(\log n)$
(b) $((a_1, b_1) \otimes (a_2, b_2)) \otimes (a_3, b_3) = (a_1 a_2 a_3,\ a_3(a_2 b_1 + b_2) + b_3)$
$(a_1, b_1) \otimes ((a_2, b_2) \otimes (a_3, b_3)) = (a_1, b_1) \otimes (a_2 a_3, a_3 b_2 + b_3) = (a_1 a_2 a_3,\ a_2 a_3 b_1 + a_3 b_2 + b_3)$ — เท่ากัน จึง associative (เป็น function composition). **ไม่ commutative**: เช่น $(2, 1) \otimes (3, 0) = (6, 3)$ แต่ $(3, 0) \otimes (2, 1) = (6, 1)$ — scan ต้องรักษาลำดับซ้าย–ขวา
(c) $P_1 = (2, 1)$; $P_2 = (2, 1) \otimes (3, 0) = (6, 3)$; $P_3 = (6, 3) \otimes (1, 4) = (6, 7)$
$x_1 = 2 \cdot 1 + 1 = 3$, $x_2 = 6 \cdot 1 + 3 = 9$, $x_3 = 6 \cdot 1 + 7 = 13$ (ตรวจ sequential: 3, $3 \cdot 3 + 0 = 9$, $9 + 4 = 13$ □)

เฉลยข้อ C5

1. **Histogram** คะแนน ($m = 101$ bins): $H[s]$ = จำนวนคนที่ได้ s
2. **Inclusive scan**$(+)$ ของ H: $C[s]$ = จำนวนคนที่ได้ $\leq s$
3. **Map**: $g_s = s$ ถ้า $C[s] \geq \lceil n/2 \rceil$ มิฉะนั้น ∞; **reduce(min)** ได้ median

$W = O(n + m) = O(n)$; S: histogram แบบ cascaded $O(m + \log n)$ (หรือ $O(\log n)$ ถ้า m ค่าคงที่), scan/reduce บน m ค่า $O(\log m) \Rightarrow S = O(\log n)$ เมื่อ $m = 101$ คงที่. ดีกว่าการ sort ($O(n \log n)$ work)

เฉลยข้อ C6

(a) $A = \langle 5, 2, 7, 4, 1, 6\rangle$ = $\langle 101, 010, 111, 100, 001, 110\rangle$

A	5	2	7	4	1	6
F (bit 0)	1	0	1	0	1	0
Down = excl-scan(not F)	0	0	1	1	2	2
P = excl-scan(F)	0	1	1	2	2	3
Index (F ? $P+3$: Down)	3	0	4	1	5	2

(จำนวน false ทั้งหมด $= 3$) $\Rightarrow R = \langle 2, 4, 6, 5, 7, 1\rangle$
(b) bit 1 บน $\langle 2, 4, 6, 5, 7, 1\rangle$: bits $= 1, 0, 1, 0, 1, 0 \Rightarrow \langle 4, 5, 1, 2, 6, 7\rangle$; bit 2: bits $= 1, 1, 0, 0, 1, 1 \Rightarrow \langle 1, 2, 4, 5, 6, 7\rangle$ □
(c) ต้อง stable เพื่อให้ลำดับที่จัดไว้จาก bit ที่ต่ำกว่าไม่ถูกทำลายในรอบถัดไป (LSD radix sort ถูกต้องได้เพราะ stability)

เฉลยข้อ C7

(a) **Map**: $t_i = (h_i - h_0)/i \rightarrow$ **exclusive scan**(max, $-\infty$) ได้ $M_i = \max_{j<i} t_j \rightarrow$ **map**: visible$_i = [t_i > M_i]$.
$W = O(n)$, $S = O(\log n)$
(b) $t = \langle -,\ 2,\ 0.5,\ 1.67,\ 1,\ 2.4,\ 2.67\rangle$ ($i = 1..6$); $M = \langle -\infty, 2, 2, 2, 2, 2.4\rangle \Rightarrow$ มองเห็นจุด $i = 1, 5, 6$

เฉลยข้อ C8

1. **Map** head flag $f_i = [i = 0 \vee A_i \neq A_{i-1}]$: $\langle 1, 0, 1, 0, 0, 1, 1, 0\rangle$
2. **Exclusive scan**($+$): $\langle 0, 1, 1, 2, 2, 2, 3, 4\rangle$ — head แต่ละตัวได้ index ปลายทาง $0, 1, 2, 3$; จำนวน run $= 4$
3. **Scatter** (compact): ค่า $V = \langle a, b, c, a\rangle$ และตำแหน่งเริ่ม $Pos = \langle 0, 2, 5, 6\rangle$, เติม $Pos[4] = n = 8$
4. **Map**: $L_k = Pos[k+1] - Pos[k] = \langle 2, 3, 1, 2\rangle$

$W = O(n)$, $S = O(\log n)$

เฉลยข้อ C9

Map: $X = \langle x, x, \ldots, x\rangle \rightarrow$ **exclusive scan**($\times$, identity 1) ได้ $\langle 1, x, x^2, \ldots\rangle \rightarrow$ map $a_i x^i \rightarrow$ **reduce**($+$).
$W = O(n)$, $S = O(\log n)$
ตัวอย่าง: $\langle 2, 2, 2, 2\rangle \rightarrow \langle 1, 2, 4, 8\rangle \rightarrow \langle 3, 0, 4, 16\rangle \rightarrow 23$ (ตรวจ: $3 + 0 + 4 + 16 = 23$ □)

เฉลยข้อ C10

Map แต่ละค่าเป็น tuple $(1, x_i, x_i^2)$ แล้ว reduce ด้วย $(n_1, s_1, q_1) \oplus (n_2, s_2, q_2) = (n_1 + n_2,\ s_1 + s_2,\ q_1 + q_2)$ — associative เพราะเป็นการบวกทีละ component (บวกเป็น associative), identity $(0, 0, 0)$. ได้ mean $\mu = s/n$, variance $= q/n - \mu^2$. $W = O(n)$, $S = O(\log n)$

ข้อควรระวัง: สูตร $q/n - \mu^2$ เกิด catastrophic cancellation เมื่อ μ ใหญ่เมื่อเทียบกับ variance; ใช้ operator แบบ Welford/Chan (n, μ, M_2) ที่ combine ได้แบบ associative แทน และ floating-point ทำให้ผลอาจต่างจาก sequential เล็กน้อยเพราะลำดับการบวกต่างกัน

ส่วน D: CUDA

เฉลยข้อ D1

(a)

```
__global__ void reduceStep(float *x, int len, int half) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < len / 2) x[i] += x[i + half];     // pairs (i, i+half)
}
void reduceGlobal(float *d_x, int n) {
    for (int len = n; len > 1; len = (len + 1) / 2) {
        int half = (len + 1) / 2;              // odd len: middle element stays
        int threads = 256, blocks = (len / 2 + threads - 1) / threads;
        reduceStep<<<blocks, threads>>>(d_x, len, half);
    }                                          // answer in d_x[0]
}
```

(b) $\log_2 2^{20} = 20$ ครั้ง. แต่ละ step ต้องรอให้ *ทุก block* เขียนผลเสร็จก่อน step ถัดไป แต่ `__syncthreads()` sync ได้เฉพาะภายใน block — การจบ kernel คือ global barrier
(c) step ที่ความยาว len อ่าน $\approx$ len ค่า เขียน len/2 ค่า: รวมอ่าน $\approx 2n$, เขียน $\approx n \Rightarrow \approx 3n \times 4$ B $= 12$ MiB เมื่อ $n = 2^{20}$. Shared-memory version อ่าน global เพียง n ค่า (4 MiB) + partial sums เล็กน้อย — traffic ลด ~ 3 เท่า และ launch น้อยลงมาก

เฉลยข้อ D2

(a) $s = 1$: thread ที่ tid คู่ทำงาน $= 128$ ตัว แต่กระจายอยู่ใน **ทั้ง 8 warps** จึงต้อง execute ทั้ง 8 warp (แต่ละ warp ใช้ lane ได้แค่ครึ่งเดียว)
(b) (1) **Warp divergence**: thread active สลับกันในทุก warp; (2) `%` เป็นคำสั่งที่ช้า; (3) ครึ่งหนึ่งของ thread ไม่ได้บวกอะไรเลยตั้งแต่ iteration แรก (โหลดแล้ว idle); (4) ทุก iteration ต้อง `__syncthreads()` แม้เหลือ warp เดียว; (5) ถ้าเปลี่ยนเป็น strided index (`idx = 2*s*tid`) จะเกิด shared memory bank conflict
(c)

```
__global__ void reduceOpt(const float *in, float *out, int n) {
    extern __shared__ float sdata[];
    unsigned tid = threadIdx.x;
    unsigned i = blockIdx.x * (blockDim.x * 2) + tid;
    float v = (i < n) ? in[i] : 0.0f;                    // (2) first add on load
    if (i + blockDim.x < n) v += in[i + blockDim.x];
    sdata[tid] = v;
    __syncthreads();
    for (unsigned s = blockDim.x / 2; s > 32; s >>= 1) {  // (1) sequential addressing
```

```
        if (tid < s) sdata[tid] += sdata[tid + s];
        __syncthreads();
    }
    if (tid < 32) {                                    // (3) last warp: shuffle
        v = sdata[tid] + sdata[tid + 32];
        for (int off = 16; off > 0; off >>= 1)
            v += __shfl_down_sync(0xffffffff, v, off);
        if (tid == 0) out[blockIdx.x] = v;
    }
}
```

(สมมุติ `blockDim.x` เป็นกำลังของ 2 และ ≥ 64)

- (1) Sequential addressing: thread active ติดกัน $\Rightarrow$ warp ทำงานเต็มหรือ idle ทั้ง warp (ลด divergence) และ thread ใน warp เข้าถึง bank ต่างกัน (ไม่มี bank conflict) — *throughput*
- (2) First add during load: ใช้ทุก thread ทำงานตั้งแต่แรก จำนวน block ลดครึ่ง — *work-efficiency / bandwidth*
- (3) Warp shuffle: ไม่ต้องผ่าน shared memory และ `__syncthreads` ในช่วง 32 ตัวสุดท้าย — *instruction overhead*
- เพิ่มเติม: grid-stride loop ให้ thread หนึ่งรวมหลาย element (coarsening/Brent) แล้ว `atomicAdd` ครั้งเดียวต่อ block

วัดผล: เวลา (cudaEvent) และ effective bandwidth เทียบ peak, Nsight: branch efficiency, shared bank conflicts

เฉลยข้อ D3

(a) bytes $= 2^{24} \times 4 = 67{,}108{,}864$; 67.1 MB$/0.5$ ms ≈ 134 GB/s $\approx 14.9\%$ ของ peak
(b) 67.1 MB$/0.085$ ms ≈ 790 GB/s $\approx 87.7\%$
(c) bytes $= 2 \times 67.1$ MB $= 134.2$ MB; $/0.4$ ms ≈ 336 GB/s $\approx 37\%$
(d) Reduce ทำ 1 การบวกต่อ 4 bytes ที่อ่าน: AI $= 0.25$ FLOP/B ซึ่งต่ำกว่า ridge point ของ GPU มาก (เช่น 10 TFLOP/s / 900 GB/s ≈ 11 FLOP/B) $\Rightarrow$ **memory-bound** ขีดจำกัดคือ bandwidth ดังนั้นตัวชี้วัดที่มีความหมายคือ % ของ peak bandwidth (GFLOP/s จะต่ำเสมอและไม่บอกว่าเหลือที่ให้ปรับอีกเท่าใด)

เฉลยข้อ D4

Bug: บรรทัด `t[tid] += t[tid-off]` อ่านและเขียน array เดียวกันใน step เดียวกัน thread tid อาจอ่าน `t[tid-off]` *หลังจาก* thread $tid - off$ เขียนค่าใหม่ของ round นี้ไปแล้ว (warp ต่างกันรันไม่พร้อมกัน; แม้ใน warp เดียวกันก็ไม่รับประกันลำดับ) ผลจึงผิดแบบไม่แน่นอน — ผิด semantics ของ forall ที่ต้องอ่านค่าเก่าทั้งหมดก่อนเขียน
วิธีแก้:

1. อ่านลง register ก่อน แล้ว sync แล้วจึงเขียน:

```
for (int off = 1; off < n; off *= 2) {
    float v = (tid >= off) ? t[tid - off] : 0.0f;
    __syncthreads();                 // all reads done
    t[tid] += v;
    __syncthreads();                 // all writes done
}
```

2. Double buffering: ใช้ shared memory $2n$ อ่านจาก buffer หนึ่ง เขียนอีก buffer แล้วสลับทุก round

เฉลยข้อ D5

(a) $n/1024 = 4096$ blocks

1. K1: scan แต่ละ block (4096 blocks) และเขียน block sum ลง S_1 (4096 ค่า)
2. S_1 มี 4096 ค่า > 1024 ต้อง recursive: K2: scan S_1 ด้วย 4 blocks ได้ block sum S_2 (4 ค่า)
3. K3: scan S_2 (1 block)
4. K4: บวก offset จาก S_2 กลับเข้า S_1 (4 blocks)
5. K5: บวก offset จาก S_1 กลับเข้าทุก element (4096 blocks)

รวม **5 kernel launches**; $W = O(n)$ ยัง work-optimal
(b) K1 อ่าน n เขียน n; K5 อ่าน n เขียน $n \Rightarrow\approx 4n$ (บวก block sums ซึ่งเล็กมาก)
แนวทางลด: (1) **Reduce-then-scan**: K1 ทำแค่ reduce ต่อ block (อ่าน n) แล้วเฟสสุดท้าย re-scan พร้อมบวก offset (อ่าน n เขียน n) $\Rightarrow\approx 3n$; (2) **Single-pass scan (decoupled look-back)**: block รอ prefix ของ block ก่อนหน้าผ่าน flag ใน global memory $\Rightarrow\approx 2n$ ซึ่งเป็นขั้นต่ำ (อ่านครั้งเดียว เขียนครั้งเดียว); (3) ให้แต่ละ thread scan หลาย element ใน register (coarsening) เพื่อลดจำนวน block sums และ sync

เฉลยข้อ D6

(a)

```
#define NUM_BINS 128
__global__ void histPrivate(const unsigned char *in, unsigned *bins, size_t n) {
    __shared__ unsigned sh[NUM_BINS];
    for (int b = threadIdx.x; b < NUM_BINS; b += blockDim.x) sh[b] = 0;
    __syncthreads();
    for (size_t i = blockIdx.x * (size_t)blockDim.x + threadIdx.x; i < n;
         i += (size_t)blockDim.x * gridDim.x)            // coalesced reads
        atomicAdd(&sh[in[i] & 0x7F], 1u);
    __syncthreads();
    for (int b = threadIdx.x; b < NUM_BINS; b += blockDim.x)
        if (sh[b]) atomicAdd(&bins[b], sh[b]);          // merge
}
```

(b) Naive: global atomic 10^8 ครั้ง; privatized: shared atomic 10^8 ครั้ง (เร็วกว่ามาก, on-chip) + global atomic $\leq 120 \times 128 = 15{,}360$ ครั้ง — ลดลง $\sim 6{,}500$ เท่า
(c) atomic บน address เดียวกันถูก **serialize**: ตัวอักษรอย่าง e, space พบบ่อยมาก ทำให้ thread จำนวนมากรอคิว bin เดียวกันใน global memory (latency สูง) throughput จึงตกมาก
Fine-tune เพิ่ม: (1) **Aggregation**: thread เก็บ bin ล่าสุดและจำนวนใน register ถ้าค่าซ้ำติดกันให้นับสะสม เมื่อค่าเปลี่ยนค่อย atomic; (2) **Per-warp sub-histograms** ใน shared memory ลด contention ภายใน block (128 bins $\times$ 4 B $\times$ #warps ยังเล็ก); (3) **Coarsening**: จำนวน block $\approx$ #SM $\times$ blocks/SM เพื่อลด merge; (4) อ่าน input แบบ vectorized (`uchar4`/`int4`) เพิ่ม bandwidth utilization. วัดผล: GB/s ของ input ที่ประมวลผล, Nsight: atomic throughput, stall reasons

เฉลยข้อ D7

(a) 4096×4 B $= 16$ KB ต่อ block
(b) shared: $\lfloor 64/16 \rfloor = 4$ blocks/SM $\Rightarrow 4 \times 256 = 1024$ threads $=$ **50%**. ปรับเป็น 512 threads/block $\Rightarrow 4 \times 512 = 2048$ threads $= 100\%$ (ถ้า register เพียงพอ) หรือใช้ counter 16-bit/8-bit ใน private histogram (saturate ที่ 127 ไม่ต้องการ 32 bits) ลด shared memory ลงได้อีก
(c) **กับดัก**: ถ้าค่าเดียวกันปรากฏ $\geq 2^{32}$ ครั้ง (input ยาวได้ถึง 2^{32}) counter 32-bit จะ overflow วนกลับเป็น 0 แล้ว clamp ตอนท้ายจะให้ 0 แทน 127 (นอกจากนี้ index ต้องเป็น `size_t` ไม่ใช่ `int`)
วิธีที่ถูกต้อง:

```
// phase 1: per-block private histogram with early saturation
if (sh[v] < 127) atomicAdd(&sh[v], 1u);   // overshoot <= threads/block, no overflow
// phase 2: merge
for (int b = threadIdx.x; b < 4096; b += blockDim.x)
    if (sh[b]) atomicAdd(&bins[b], min(sh[b], 127u));  // global <= 127 * #blocks
// phase 3 (separate kernel, after all blocks): clamp
bins[b] = min(bins[b], 127u);
```

การตรวจ `sh[v] < 127` ก่อน atomic อาจเกิน 127 ได้บ้าง (race) แต่เกินได้ไม่เกินจำนวน thread ใน block จึงไม่ overflow; หรือใช้ `atomicCAS` loop เพื่อ saturate แบบ exact. ค่าใน global ไม่เกิน $127 \times$ #blocks จึงปลอดภัยแล้ว clamp ใน kernel ที่สอง (ต้องแยก kernel เพราะต้องรอทุก block merge เสร็จ)

เฉลยข้อ D8

กรณี	threads	regs	shared	max	blocks/SM	occupancy (คอขวด)
(a)	8	8	8	32	8	$2048/2048 = 100\%$
(b)	8	**4**	8	32	4	$1024/2048 = 50\%$ (registers)
(c)	8	8	**2**	32	2	$512/2048 = 25\%$ (shared)
(d)	64	128	∞	**32**	32	$1024/2048 = 50\%$ (max blocks)

วิธีคำนวณ เช่น (a): threads $2048/256 = 8$; regs $65536/(32 \cdot 256) = 8$; shared $64/8 = 8$. (b): regs $65536/(64 \cdot 256) = 4$. (c): shared $\lfloor 64/24 \rfloor = 2$. (d): threads $2048/32 = 64$, regs $65536/(16 \cdot 32) = 128$ แต่ติดเพดาน 32 blocks/SM. บทเรียน (d): block เล็กเกินไปชนเพดานจำนวน block ต่อ SM ควรใช้ block ≥ 64–128 threads

เฉลยข้อ D9

Warp ต้องการ $32 \times 4 = 128$ B

- (a) $A[i]$: byte 0–$127 = 4$ sectors $\Rightarrow$ **100%**
- (b) $A[2i]$: ครอบคลุม 256 B $= 8$ sectors ใช้จริง 128 B $\Rightarrow$ **50%**
- (c) $A[32i]$: แต่ละ thread อยู่คนละ 128 B $\Rightarrow$ 32 sectors (1024 B) ใช้จริง 128 B $\Rightarrow$ **12.5%**
- (d) $A[i+1]$: byte 4–131 คร่อม 5 sectors (160 B) $\Rightarrow 128/160 =$ **80%** (misaligned)
- (e) AoS stride 12 B: ครอบคลุม $32 \times 12 = 384$ B $= 12$ sectors ใช้จริง 128 B $\Rightarrow\approx$ **33%**. แก้โดยเปลี่ยนเป็น **Structure of Arrays** (`float x[N], y[N], z[N]`) ได้ 100%

เฉลยข้อ D10

bank $=$ (index) mod 32 (สำหรับ float 4 B)

- (a) $sh[t]$: bank $= t$ ต่างกันทุกตัว $\Rightarrow$ ไม่มี conflict
- (b) $sh[2t]$: bank $= 2t$ mod 32; t กับ $t+16$ ชน bank เดียวกัน $\Rightarrow$ **2-way**
- (c) $sh[32t]$: ทุกตัวอยู่ bank 0 $\Rightarrow$ **32-way**
- (d) $sh[33t]$: bank $= 33t$ mod $32 = t \Rightarrow$ ไม่มี conflict (นี่คือหลักการ padding)

Blelloch ใช้ index `offset*(2*tid+1)-1` ที่มี stride เป็น 2^{d+1} เมื่อ d เพิ่ม stride เป็น 2, 4, 8, ..., 32 ทำให้เกิด 2-, 4-, ..., 32-way conflict. แก้ด้วย **padding**: `idx + (idx >> 5)` (เว้น 1 ช่องทุก 32 ช่อง) ทำให้ stride ที่เป็นพหุคูณของ 32 กระจายไปคนละ bank

เฉลยข้อ D11

(a) **Reduce**$(+)$ หา sum $\rightarrow \mu =$ sum$/n \rightarrow$ **map** flag $F_i = [x_i > \mu] \rightarrow$ **exclusive scan**$(+)$ ของ $F \rightarrow$ **scatter** ($R[X_i] \leftarrow x_i$ ถ้า F_i). $W = O(n)$, $S = O(\log n)$ (ต่อกันจึงบวก step: $O(\log n) + O(1) + O(\log n) + O(1)$), work-optimal

(b) ตัวอย่างการออกแบบ (3 kernels):

1. K1 **reduce**: grid-stride + warp shuffle + `atomicAdd` ได้ sum (อ่าน n)
2. K2 **flag + block-level scan**: แต่ละ block คำนวณ flag ใน register, นับจำนวนที่ผ่านใน block (เขียน block count) — อ่าน n
3. K3 **scan block counts + scatter**: ได้ offset ของ block แล้วแต่ละ block ทำ scan ภายใน block อีกครั้งและเขียนเฉพาะตัวที่ผ่าน — อ่าน n เขียน k

(scan ของ block counts ทำใน kernel เล็กแยก หรือใช้ decoupled look-back)

(c) Fine-tune ideas:

- **Kernel fusion** (utilize bandwidth): ไม่เขียน array flag/scan ขนาด n ลง global — คำนวณ flag ใหม่จาก x_i ใน K2/K3 (เพราะ recompute ถูกกว่าอ่าน/เขียน memory); traffic $\approx 3n$ อ่าน $+ k$ เขียน แทน $\approx 7n$ ของแบบแยกทุกขั้น
- **Warp-level primitives** (throughput): ใช้ `__ballot_sync` + `__popc` ทำ scan ของ flag ภายใน warp แทน shared-memory scan
- **Coarsening**: thread หนึ่งประมวลผลหลาย element (เช่น 4–8 ผ่าน `float4`) ลด overhead ของ sync และจำนวน block
- **Coalesced write**: รวม output ของ block ใน shared memory ก่อนแล้วเขียนออกแบบต่อเนื่อง
- **วัดผล**: effective bandwidth $= (3n \cdot 4 + k \cdot 4)/t$ เทียบ peak; ถ้าเกิน $\sim$80% ถือว่าใกล้ขีดจำกัดแล้ว; Nsight: `dram__throughput`, achieved occupancy, warp stall reasons